Movie Review Agent

Technical Specification

Groq API · llama-3.3-70b-versatile (Critic) · mixtral-8x7b-32768 (Advocate)

Version 1.0 · April 2026

1. Overview

CineGlow is a Streamlit web application that lets users search any film and watch two language models — a Critic and an Advocate — debate its quality across four structured rounds before converging on a calibrated score. All inference runs through the Groq API, with metadata sourced from the OMDb API in real time.

Entry point	app.py — Streamlit UI and orchestration
Debate engine	agent/sentiment.py — analyze_movie()
Inference API	Groq API (https://api.groq.com/openai/v1)
Metadata API	OMDb API (https://www.omdbapi.com)
UI theme	Dark Velvet Cinema — mahogany-black, copper accents
Config	.streamlit/secrets.toml (gitignored)

2. System Architecture

The system follows a three-layer architecture: a Streamlit presentation layer, an agent orchestration layer, and two external API integrations.

```
cineglow/
├── app.py                    # Entry point — UI + orchestration
├── agent/
│   └── sentiment.py          # Debate engine — analyze_movie()
├── .streamlit/
│   └── secrets.toml          # API keys (gitignored)
├── architecture.svg          # System architecture diagram
├── README.md
└── requirements.txt
```

2.1 Data Flow

```
User types query
      │
      ▼
search_movies()  ──▶ OMDb ?t= (exact) + ?s= (fuzzy)
      │
      ▼
Poster Grid  ──▶ User selects film
      │
      ▼
fetch_movie_by_id()  ──▶ OMDb ?i= (full detail)
      │
      ▼
analyze_movie()  ──▶ Critic  (Groq: llama-3.3-70b-versatile) × 4 rounds
                 ──▶ Advocate (Groq: mixtral-8x7b-32768)      × 4 rounds
                 ──▶ Returns: {debate_transcript, debate_summary,
                           final_score, themes, scoring_basis}
      │
      ▼
Streamlit render pipeline
(poster · meta · themes · overview · agenda · summary · scores · transcript)
```

3. API Configuration

3.1 Groq API

CineGlow uses the Groq API for all LLM inference. The Groq client is initialised via the OpenAI-compatible SDK with a custom base URL.

```
from openai import OpenAI

client = OpenAI(
    api_key=st.secrets["GROQ_API_KEY"],
    base_url="https://api.groq.com/openai/v1"
)
```

GROQ_API_KEY	Groq API key from console.groq.com
Critic model	llama-3.3-70b-versatile
Advocate model	mixtral-8x7b-32768
Base URL	https://api.groq.com/openai/v1
SDK	openai (OpenAI-compatible client)
Max tokens	1,000 per completion call

3.2 OMDb API

`OMDB_API_KEY`	OMDb key from omdbapi.com/apikey.aspx
`Exact search`	https://www.omdbapi.com/?t={title}
`Fuzzy search`	https://www.omdbapi.com/?s={query}
`Detail fetch`	https://www.omdbapi.com/?i={imdb_id}
`Free tier limit`	1,000 requests/day

3.3 secrets.toml

```
# .streamlit/secrets.toml  (gitignored — never commit)
OMDB_API_KEY      = "your_omdb_key"
GROQ_API_KEY      = "your_groq_key"
```

4. Agent Module — agent/sentiment.py

4.1 Exports

`analyze_movie(raw_reviews)`	Runs 4-round debate, returns result dict
`MODEL_CRITIC`	"llama-3.3-70b-versatile" (Groq)
`MODEL_ADVOCATE`	"mixtral-8x7b-32768" (Groq)

4.2 Result Schema

```
{
  "debate_transcript": [
    {"role": str, "model": str, "text": str},
    ...
  ],
  "debate_summary":  str,    # 2-3 sentence neutral synthesis
  "final_score":     str,    # e.g. "7.5/10"
  "themes":          [str], # 3-6 thematic keywords
  "scoring_basis":   str     # methodology note
}
```

4.3 Debate Rounds

Round	Agent	Focus
1	Critic → Advocate	Initial critical assessment vs. contrarian defence
2	Critic → Advocate	Deep dive: craft, cinematography, character work
3	Critic → Advocate	Rebuttals: pacing, cultural legacy, audience reception
4	Critic → Advocate	Nuanced synthesis and calibrated final score derivation

5. app.py — Orchestrator + UI

5.1 Key Functions

`search_movies(query)`	Exact + fuzzy OMDb search, merged and deduped (up to 6 results)
`fetch_movie_by_id(imdb_id)`	Full movie detail including Rotten Tomatoes rating
`ring_svg(pct, ...)`	Generates animated SVG score ring
`score_card(svg, label)`	Wraps ring in labelled card HTML

5.2 Session State Keys

`cached_movie`	Last fully fetched movie detail dict
`cached_result`	Last analyze_movie() result dict
`search_results`	List of OMDb search result dicts
`selected_imdb_id`	IMDb ID of currently selected film
`last_typed`	Last search query string (debounce)
`search_error`	Error message string or None

6. Design System — Dark Velvet Cinema

All colour tokens live in the C dict at the top of app.py. To retheme, update C values and the @import font URL.

Token Group	Token	Value
Surfaces	bg_deep	#120c09 (mahogany-black)
Surfaces	bg_card	#1e1510
Surfaces	bg_panel	#2a1c14
Text	text_cream	#f0e4d4
Text	text_warm	#c9b098
Text	text_muted	#7a5c42
Accent	copper	#e8833a
Accent	copper_light	#f09050
Accent	ember	#c8603a
Borders	border_mid	#4a3020
Borders	border_dark	#2e1e12
Fonts	heading	Playfair Display (Google Fonts)
Fonts	body	Outfit (Google Fonts)

7. Requirements & Installation

7.1 Python Dependencies

```
# requirements.txt
streamlit>=1.32
requests>=2.31
openai>=1.0      # OpenAI-compatible client for Groq API
```

7.2 Quick Start

```
# 1. Install dependencies
pip install streamlit requests openai
# 2. Create .streamlit/secrets.toml with your API keys
# OMDB_API_KEY  = "your_omdb_key"
# GROQ_API_KEY  = "your_groq_key"
# 3. Run
streamlit run app.py
# Open http://localhost:8501
```

8. Feature Summary

Feature	Module	Description
Smart Search	app.py	Exact + fuzzy OMDb search, deduped, up to 6 poster results
AI Debate Engine	agent/sentiment.py	4-round Critic vs. Advocate via Groq API
Score Ring Charts	app.py → ring_svg()	Animated SVG rings for AI verdict, IMDb, and RT scores
Dark Velvet UI	app.py → C dict	Molten copper glows, Playfair Display serif, custom CSS
Debate Agenda Strip	app.py	Full-width 4-cell horizontal layout
Theme Chips	agent/sentiment.py	AI-extracted genre/thematic tags per film
Scoring Methodology	agent/sentiment.py	Expandable scoring basis and calibration note

9. Success Criteria

`Debate quality`	Each round must advance the argument — no repeated claims
`Score calibration`	AI verdict correlates meaningfully with IMDb and RT baselines
`Transparency`	Users can inspect every scoring criterion and debate turn
`Latency`	Full four-round debate and verdict within 30 seconds
`Metadata accuracy`	Director, cast, genre, ratings via OMDb API in real time
`API correctness`	Groq API with correct model strings: llama-3.3-70b-versatile / mixtral-8x7b-32768

10. Roadmap

- Streaming debate output (real-time token display via Groq streaming)
- Watchlist / favourites with session or cloud persistence
- Unit tests for search_movies() and fetch_movie_by_id()
- Dark / light mode toggle
- Director / actor filmography deep-dives